Multiplayer Treasure Hunt – Brief Summary

Project github link - https://github.com/Pranika-7/treasurehunt.git

1. Project Overview
   - Multiplayer game where players find and collect hidden treasures.
   - Each treasure can be claimed by only one player.
   - Players earn points based on treasure values.
2. Technology
   - Java + Spring Boot
   - Spring Data JPA / Hibernate
   - MySQL
   - REST APIs
3. Architecture
   - Controller: Handles HTTP requests.
   - Service: Contains game logic.
   - Repository: Handles database operations.
   - Database: Stores players, treasures and scores.
4. Database Tables
   - Players – ID, username, score.
   - Treasures – ID, coordinates, points, claimed status.
   - Game Sessions – ID, status.
5. REST APIs
   - POST /players – Register player.
   - GET /treasures – View treasures.
   - POST /treasures/{id}/claim – Collect treasure.
   - GET /leaderboard – View rankings.
6. Concurrency
   - Multiple players may claim the same treasure simultaneously.
   - Uses database-level atomic updates.
   - Only one player succeeds.
   - Others receive HTTP 409 Conflict.
7. Testing
   - JUnit 5
   - ExecutorService
   - CountDownLatch
   - Tests simultaneous claims and leaderboard accuracy.
8. Important Concepts
   - Spring Boot
   - REST APIs
   - JPA and Hibernate
   - MySQL
   - Transactions (@Transactional)
   - Concurrency
   - Race conditions
   - Database locking
   - ACID properties

Building a multiplayer treasure-hunting backend where Spring Boot manages game operations and MySQL ensures that treasures cannot be claimed twice.